Faculdade de Tecnologia de Sorocaba
Tecnologia em Análise e Desenvolvimento de Sistemas

NICOLLY LEMOS DA SILVA

DOCUMENTAÇÃO DE SOFTWARE:
SISTEMA DE AGENDAMENTO ONLINE

SOROCABA/SP
2026

1. Objetivo

O sistema foi pensado para pequenos negócios de serviços, como salões, barbearias, estúdios de estética e manicures, que querem ter o próprio agendamento online sem depender de plataformas de terceiros. O cliente entra no site da empresa, escolhe o serviço, o profissional e o horário, e garante a vaga pagando um sinal pelo Pix. Do outro lado, a equipe tem um painel único para cuidar da agenda, dos horários de trabalho, do estoque de produtos, de um programa de fidelidade e dos relatórios do negócio.

2. Problema

Boa parte desses negócios ainda marca horário pelo WhatsApp ou num caderno. Na prática, isso significa muita troca de mensagem até achar um horário livre, clientes que marcam e não aparecem (porque não existe nenhum compromisso financeiro) e, de vez em quando, dois atendimentos no mesmo horário para a mesma pessoa.

Além da agenda, falta controle sobre o resto. O dono raramente sabe quanto de produto cada serviço consome, qual é a margem real ou quando o estoque vai acabar. Cartões de fidelidade em papel se perdem, e faturamento, comissões e ocupação da agenda acabam virando estimativa. As ferramentas prontas do mercado resolvem parte disso, mas cobram por profissional, colocam a marca delas no site e quase nunca juntam agenda, sinal, estoque e fidelidade no mesmo lugar.

3. Escopo

Esta versão cobre o site público da empresa (personalizável e com prévia antes de publicar), o agendamento online sem precisar criar conta, a cobrança do sinal via Pix com confirmação automática, o painel da equipe com agenda, serviços, horários e estoque, o programa de fidelidade, os relatórios com exportação em CSV e PDF, os lembretes por e-mail e o controle de acesso por perfil.

Ficaram de fora, por enquanto: atender mais de uma empresa na mesma instalação, aplicativo mobile nativo (o site e o painel funcionam bem no celular), lembretes por WhatsApp, emissão de nota fiscal e pagamento do valor total online. Só o sinal é cobrado pelo sistema; o restante é registrado quando o atendimento é concluído.

4. Requisitos funcionais

Os requisitos foram numerados de RF01 a RF55 e o mesmo código aparece nos comentários do código-fonte, o que facilita encontrar onde cada um foi implementado. A tabela abaixo resume os requisitos por área.

Área	Códigos	O que o sistema faz
Acesso	RF01–RF05	Login com e-mail e senha e 2FA opcional, convite da equipe por link de uso único, recuperação de senha, acesso do cliente por código enviado por e-mail e bloqueio imediato de quem for desativado

Área	Códigos	O que o sistema faz
Empresa e site	RF06–RF11	Cadastro da empresa, personalização do site (cores, fontes, logo, banner, textos e seções), prévia antes de publicar e botão que leva direto ao agendamento do serviço
Serviços e equipe	RF12–RF15	Cadastro de serviços com foto, definição de quem executa cada um (com preço e duração próprios), ficha técnica de produtos e cálculo de custo e margem
Horários	RF16–RF20	Modelos de jornada, horários por profissional, folgas e feriados, bloqueios manuais e cálculo automático dos horários livres
Agendamento	RF21–RF27	Agendamento pelo site e pelo painel, antecedência mínima e máxima, remarcação e cancelamento, controle de status e lembretes 24 h e 2 h antes
Sinal e Pix	RF28–RF37	Sinal fixo ou percentual, Pix com QR Code, confirmação automática, expiração do horário não pago, política de devolução e cadastro protegido da conta que recebe
Estoque	RF38–RF42	Cadastro de produtos, entradas e saídas com responsável, baixa automática ao concluir o atendimento, alertas de estoque mínimo e validade, e custo médio
Fidelidade	RF43–RF46	Programa de pontos ou cartão de selos, recompensas, pontos creditados só na conclusão e extrato para o cliente
Relatórios	RF47–RF55	Faturamento, sinais, ocupação da agenda, rentabilidade por serviço, estoque, clientes, fidelidade e comissões, com exportação em CSV e PDF

5. Requisitos não funcionais

Código	Categoria	Requisito
RNF01	Integridade	Nunca deve existir dois agendamentos no mesmo horário para o mesmo profissional, e quem garante isso é o próprio banco de dados, mesmo com pedidos chegando ao mesmo tempo
RNF02	Segurança	Senhas guardadas com argon2id, mínimo de 10 caracteres e checagem contra senhas vazadas
RNF03	Segurança	Sessão com token de curta duração (15 min) e renovação por cookie protegido
RNF04	Segurança	Cada perfil (Admin, Gerente, Funcionário) só acessa o que tem permissão, e a checagem acontece sempre no servidor
RNF05	Segurança	Dados sensíveis, como as credenciais do Pix, ficam criptografados no banco
RNF06	Segurança	Limite de tentativas em login, agendamento e Pix, com bloqueio progressivo depois de 5 erros de senha
RNF07	Confiabilidade	O valor do sinal é sempre calculado no servidor e o webhook do pagamento é conferido antes de confirmar qualquer coisa
RNF08	Auditoria	Ações sensíveis ficam registradas, e o histórico de estoque e de pontos não pode ser alterado depois de gravado
RNF09	Usabilidade	Interface em português e responsiva, usável no celular tanto pelo cliente quanto pela equipe
RNF10	Manutenibilidade	TypeScript nos dois lados, código organizado por módulo e testes automáticos rodando a cada envio ao GitHub

6. Tecnologias utilizadas

Camada	Tecnologias
Backend	Node.js 22, Express 5, TypeScript, Prisma, Zod
Banco de dados	PostgreSQL 16
Filas e cache	Redis 7 e BullMQ
Frontend	React 19, Vite, Tailwind CSS 4, TanStack Query, React Router
Segurança	argon2, JWT com EdDSA, 2FA por TOTP, Helmet, criptografia AES-256-GCM
Pagamentos	Mercado Pago (Pix) e um gateway simulado para desenvolvimento
Testes e CI	Vitest, Supertest e GitHub Actions
Ambiente	Docker Compose para subir o Postgres e o Redis localmente

7. Arquitetura da aplicação

O sistema segue o modelo cliente-servidor. O frontend é uma aplicação React que funciona tanto como site público da empresa quanto como painel da equipe, e conversa com uma API REST em Express. A API guarda tudo no PostgreSQL e usa o Redis para duas coisas: controlar o limite de requisições e manter uma fila de tarefas em segundo plano.

Toda requisição passa primeiro por uma camada de proteção, que verifica cabeçalhos de segurança, limite de acessos, login, permissão do usuário e formato dos dados enviados. Só depois disso ela chega às regras de negócio, que ficam separadas por módulo e falam com o banco pelo Prisma. Os erros voltam sempre num formato padrão, sem expor detalhes internos.

A fila de tarefas cuida do que não depende de alguém clicar em nada: expirar agendamentos cujo sinal não foi pago, enviar lembretes, avisar sobre estoque baixo ou produto vencendo e vencer pontos de fidelidade antigos. Se o Redis cair, a própria API faz uma varredura a cada poucos minutos para que nenhum horário fique preso.

O fluxo mais importante é o do agendamento com sinal. O cliente escolhe serviço, profissional e horário, e a API grava o agendamento como "aguardando pagamento". Nesse momento o horário já fica reservado, e o banco recusa qualquer outro agendamento que se sobreponha. Em seguida a API gera a cobrança Pix no Mercado Pago e devolve o QR Code. Quando o cliente paga, o Mercado Pago avisa a API por webhook, a API confere o pagamento direto no gateway e só então muda o status para "confirmado". Se o prazo passar sem pagamento, o agendamento expira e o horário volta a ficar livre.

8. Organização dos componentes

O projeto é um monorepo com três pacotes: apps/api (a API), apps/web (site e painel) e packages/shared (tipos usados pelos dois lados).

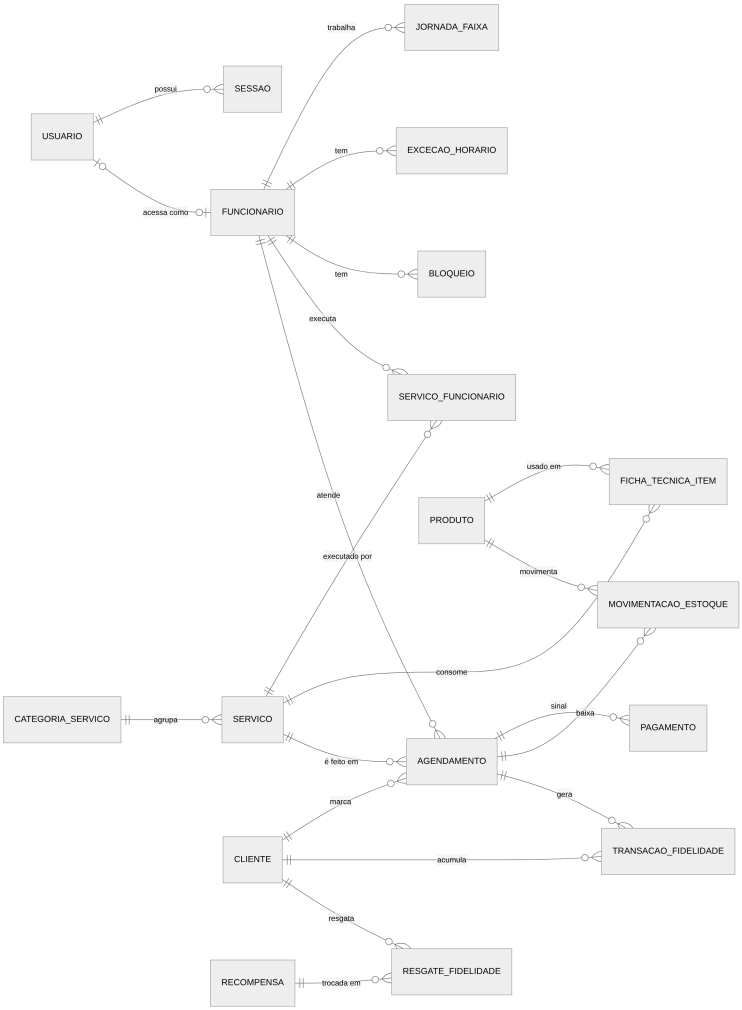
Na API, cada domínio do negócio tem sua própria pasta dentro de src/modules, com as rotas, as validações e as regras de negócio juntas. Os principais módulos são auth (login, 2FA e permissões), servicos, funcionarios, horarios, agendamentos, pagamentos, estoque, fidelidade, personalizacao e relatorios. Fora dos módulos ficam os middlewares de segurança, a fila de tarefas (src/jobs) e utilitários como conexão com banco, criptografia, e-mail e logs (src/lib).

No frontend, as telas ficam em src/pages, divididas em public (site, agendamento e área do cliente), admin (painel do administrador), staff (área do profissional) e agenda. Os componentes reaproveitáveis ficam em src/components, e o acesso à API está centralizado em src/lib.

9. Banco de dados

O banco é PostgreSQL, modelado com o Prisma e versionado por migrations em apps/api/prisma/migrations. São 32 tabelas, agrupadas assim:

Domínio	Tabelas principais
Empresa e site	empresas, personalizacao, arquivos, posts_instagram
Acesso	usuarios, sessoes, tokens_uso, logs_auditoria
Equipe e serviços	funcionarios, servicos, categorias_servico, servicos_funcionarios
Horários	jornada_faixas, modelos_jornada, excecoes_horario, bloqueios
Agendamento	clientes, agendamentos, codigos_cliente
Pagamentos	pagamentos, webhook_eventos, recebedor
Estoque	produtos, movimentacoes_estoque, fichas_tecnicas
Fidelidade	programa_fidelidade, recompensas, transacoes_fidelidade, resgates_fidelidade



Algumas regras ficam no próprio banco, e não só no código. A principal é a trava da agenda: uma constraint do PostgreSQL que impede dois agendamentos ativos do mesmo profissional em horários que se sobrepõem. Isso foi testado com 20 pedidos

simultâneos para o mesmo horário, e só um foi aceito. O histórico de estoque e o extrato de pontos são protegidos por triggers que impedem alteração ou exclusão. Os valores em dinheiro são guardados em centavos, e o agendamento guarda uma cópia do preço e da duração do serviço no momento em que foi feito, para que mudanças futuras não alterem o que já foi combinado.

10. APIs

A API segue o padrão REST, responde em JSON e todas as rotas começam com /api/v1. Elas se dividem em públicas (site e agendamento, usadas pelo cliente), autenticação, administrativas (/admin, para gerente e administrador), da equipe (/staff, em que o profissional só enxerga os próprios dados) e o webhook de pagamento. A lista completa de endpoints está no arquivo docs/API.md do repositório.

11. Variáveis de ambiente

As variáveis ficam no arquivo apps/api/.env, que nunca vai para o GitHub (há um .env.example como modelo). A API confere todas ao iniciar e não sobe se faltar alguma.

Variável	Para que serve
DATABASE_URL	Conexão com o PostgreSQL
REDIS_URL	Conexão com o Redis
APP_URL e CORS_ORIGIN	Endereço do front, usado nos links enviados por e-mail e para liberar o acesso à API
JWT_PRIVATE_KEY e JWT_PUBLIC_KEY	Chaves de assinatura do login, geradas com npm run gerar-chaves
ENCRYPTION_KEY	Chave usada para criptografar dados sensíveis
API_URL	Endereço público da API, para onde o Mercado Pago envia o webhook
ARMAZENAMENTO	Onde ficam as imagens: local em desenvolvimento ou S3 em produção

12. Testes e deploy

O projeto tem cerca de 190 testes automatizados com Vitest. Os unitários cobrem partes isoladas, como o cálculo de horários livres e a criptografia. Os de integração simulam fluxos inteiros contra um banco real, do login ao relatório, incluindo o teste de concorrência da agenda e tentativas de acessar dados de outro usuário. Todos rodam com npm test, e a cada envio ao GitHub o GitHub Actions procura senhas esquecidas no código, verifica vulnerabilidades nas dependências, checa os tipos e roda os testes.

Para o deploy, o caminho previsto é uma VPS Linux com Nginx. O PostgreSQL e o Redis rodam no servidor sem acesso externo. O projeto é compilado com npm run build, as migrations são aplicadas com npx prisma migrate deploy e a API fica rodando com o pm2. O Nginx serve o front já compilado, repassa as chamadas de /api para a API e cuida do HTTPS. Por fim, a conta do Mercado Pago é cadastrada pelo painel e o webhook é apontado para o endereço público da API. O passo a passo completo está no README do repositório.